



FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

TEST 3 (Intersystem communication)

Memorandum

DATE: 14 MAY 2026

TIME: 1 Hour

EXAMINER: BOAMAH-ABU, CHARLES (PhD)

TOTAL MARKS: 30

NO. OF PAGES: 4 (Including cover page)

INSTRUCTIONS/REQUIREMENTS

1. *This examination is closed book*
2. *Answer all questions*
3. *Where stated "Briefly answer", please give a short answer*
4. *Where stated "Write the missing parts" of code, please write only the missing parts*
5. *Ensure your handwriting is readable*
6. **NOTE: Java is case sensitive**

Question 1

3 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false)

1. In client-server applications, the client listens for a request.
2. Both SOAP and REST are used in web services.
3. Resource Oriented Architecture is a design pattern for distributed applications.

1. F 2. T 3. T

Question 2

4 marks

Briefly explain:

1. Why is distributed applications important in Integrative programming. (2)

Any correct answer: e.g., distributed applications are standalone (processes and data) and work together for a common goal; this supports Integrative programming

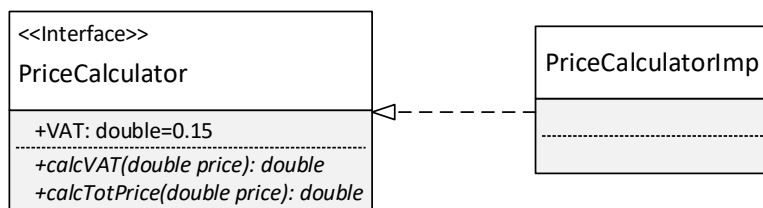
2. Why is web service important in Integrative programming (2)

Any correct answer: web services are platform-independent, which supports Integrative programming

Question 3 (RMI programming)

23 marks

You are required to write an RMI application to calculate the VAT payable and total price (i.e., price + VAT payable) on items in a shop. Below is the remote interface and the implementation class.



- **VAT**: the VAT charged on an item; a constant equal to 0.15
- **calcVAT(double price)**: takes the price of an item as a parameter and returns the VAT payable on the item
- **calcTotPrice(double price)**: takes the price of an item as a parameter and returns the total price (i.e., price + VAT chargeable)

1. Complete the code for the remote interface. (7)

```
public _____{
    _____
    _____
    _____
}
```

Answer:

```
public interface PriceCalculator extends Remote✓✓{  
    double VAT=0.15; ✓  
    double calcVAT(double price) throws Exception; ✓✓  
    double calcTotPrice(double price) throws Exception; ✓✓  
}
```

2. Complete the implementation class of the remote interface as follows (write only the **question number** and the **missing part**):

- a. Write the missing part of the class header (2)
- b. Write the empty constructor of the class (2)

```
public class PriceCalculatorImp _____ 2a _____{  
  
    public _____ 2b _____{  
  
    }  
  
    @Override  
    public double calcVAT(double price) {  
        return price * VAT;  
    }  
  
    @Override  
    public double calcTotPrice(double price) {  
        return price + calcVAT(price);  
    }  
  
}
```

```
public class PriceCalculatorImp extends UnicastRemoteObject implements PriceCalculator✓✓ {  
  
    public PriceCalculatorImp() throws Exception✓✓{  
    }  
  
    ...  
    ...  
  
}
```

3. A class **ServerTester** (below) creates a registry (called **r1**) on port **2026** to register an instance (called **pci**) of the remote class **PriceCalculatorImp** to a unique name “**calculator**”. Complete the code (write only the **question number** and the **missing part**) as follows:

- a. Create an instance of the remote class **PriceCalculatorImp** (1)
- b. Create the registry **r1** on port **2026** (2)
- c. Register the instance **pci** with the unique name “**calculator**” (2)

```
public class ServerTester {  
  
    public static void main(String[] args) throws Exception{  
        PriceCalculatorImp pci=_____a_____;
```

```

        Registry r1= _____ b _____;
        _____ c _____;
    }
}

```

Answer:

```

public class ServerTester {

    public static void main(String[] args) throws Exception{
        PriceCalculatorImp pci=new PriceCalculatorImp();✓
        Registry r1=LocateRegistry.createRegistry(2026);✓✓
        r1.rebind("calculator", pci);✓✓
    }
}

```

4. A class **ClientTester** (below) “looks up” the reference to the instance registered in (3) by the class **ServerTester** above and calls the remote methods. Complete the code (write only the **question number** and the **missing part**) to:

- a. Obtain the reference to “**calculator**”. The name of the server is “**localhost**” (3)
- b. (If the price of an item is R1000), call the method which returns and displays on the screen the,
 - i. VAT chargeable (2)
 - ii. Total price (2)

```

public class ClientTester {
    public static void main(String[] args) throws Exception{
        PriceCalculator pci = (PriceCalculator)_____ a _____;
        double price=1000;
        System.out.println("VAT is R" + _____ b(i) _____);
        System.out.println("Total Price (including VAT) R=" + _____ b(ii) _____);
    }
}

```

Answer:

```

public class ClientTester {
    public static void main(String[] args) throws Exception{
        PriceCalculator pc=(PriceCalculator) Naming.Lookup✓ ("rmi://localhost:2026/calculator");✓✓
        double price=1000;
        System.out.println("VAT is R" + pc.calcVAT(price));✓✓
        System.out.println("Total Price (including VAT) R=" + pc.calcTotPrice(price));✓✓
    }
}

```